PYTHON 90 DAYS CHALLENGE

DAY 11 — LISTS IN PYTHON

Source coverage: Python Notes X CodeWithNishchal, pages 125–129.

Day Goal

Understand how to create lists, access items using positive and negative indexes, update values, and use common list methods and built-in functions.

1. What is a List?

A **list** is a Python data structure used to store multiple values in one variable. Lists are written inside square brackets `[]`, and items are separated by commas.

Important Properties

Property	Meaning	Example
Ordered	Items keep their positions	items[0]
Mutable	Items can be changed	items[1] = 'New'
Duplicates allowed	Same value can appear more than once	[10, 10, 20]
Mixed data types	Can hold different data types	['Aman', 20, True]
Indexed	Each item has an index	0, 1, 2, ...

2. Creating Lists

```
numbers = [10, 20, 30, 40]
names = ["Aman", "Riya", "Kabir"]
mixed = ["Python", 100, 99.5, True]
empty = []
nested = [[1, 2], [3, 4]]
letters = list("ABC")

print(numbers)
print(names)
print(mixed)
print(empty)
print(letters)
```

A list can contain numbers, strings, Boolean values, mixed values, or other lists. An empty list is written as [].

3. Accessing List Values

Use an index number inside square brackets to access an item. Python indexing starts from **0**. Positive indexing starts at the left; negative indexing starts from the right.

```
students = ["Aman", "Riya", "Kabir", "Neha"]

print(students[0])   # Aman
print(students[1])   # Riya
print(students[-1])  # Neha
print(students[-2])  # Kabir
```

Code	Meaning	Result
students[0]	First item	"Aman"

Code	Meaning	Result
students[1]	Second item	"Riya"
students[-1]	Last item	"Neha"
students[-2]	Second-last item	"Kabir"

4. Updating List Values

Lists are mutable, so an existing item can be changed using its index. The index must already exist in the list.

```
marks = [70, 80, 90]
marks[1] = 85
print(marks)
# Output: [70, 85, 90]

values = [70, 80, 90, 60]
values[1] = 88
values[2] = 95
print(values)
# Output: [70, 88, 95, 60]
```

5. Important List Methods

Method	Purpose	Example
append(x)	Adds one item at the end	items.append(40)
extend(iterable)	Adds multiple items	items.extend([50, 60])
insert(i, x)	Adds an item at a specific index	items.insert(1, 15)
remove(x)	Removes the first matching value	items.remove(20)
pop(i)	Removes and returns item at index i	items.pop(1)
clear()	Removes all items	items.clear()
index(x)	Returns the index of a value	items.index(30)
count(x)	Counts occurrences of a value	items.count(10)
sort()	Sorts the list in ascending order by default	items.sort()
reverse()	Reverses the list order	items.reverse()
copy()	Creates a shallow copy	new = items.copy()

Example Using List Methods

```
numbers = [10, 20, 30]
numbers.append(40)
numbers.insert(1, 15)
numbers.remove(20)
print(numbers)
# Output: [10, 15, 30, 40]
```

6. Useful Built-in Functions with Lists

Function	Purpose	Example
len(list)	Counts the number of items	len(numbers)
sum(list)	Adds numeric items	sum(numbers)
min(list)	Returns the smallest value	min(numbers)

Function	Purpose	Example
max(list)	Returns the largest value	max(numbers)
sorted(list)	Returns a sorted copy; original list is unchanged	sorted(numbers)